# Hadoop Lab

## 1 Environment

We will run a toy Hadoop application on a single node. A better option would have been to use the docker image from bde2020 and deploy a cluster of containers with docker-compose. But this goes beyond the scope of this project and the machines at university do not support docker-compose. Therefore we install a lighter image `hadoop-lab msfasha/hadoop-petra` which runs directly hadoop within a single container.
As indicated in the image's webpage, it should be launched as follows :

```
docker run -it -p 9870:9870 -p 8088:8088 --name hadoop-lab msfasha/hadoop-petra
```

### 1.1 Setting up Hadoop in pseudo-distributed mode

In this mode we use one Java process per Hadoop daemon (unline *standalone* mode), but we do not have a real cluster.

You can observe in the log messages that print after launching the process that the container runs a namenode, a secondary namenode, etc. But all nodes are located on the same container.

1. The Hadoop executables are located in directory `$HADOOP_HOME`. Check where this directory is located by printing the environment variables, and **make it the current directory if needed but this should already be the case with this docker image**.
2. Check some configuration information : `bin/hdfs dfsadmin -report` and `bin/hdfs getconf -namenodes`
3. Have a quick look at configuration files : `etc/hadoop/core-site.xml` and `etc/hadoop/hdfs-site.xml`
   - check that we are by default using HDFS, and not the local filesystem as in standalone mode (hdfs dfs relies on fs.defaultFS to identify the HDFS cluster it manipulates, whereas bin/hadoop fs uses it to identify the more general file system it manipulates).
   - check the default replication factor.
   - check the directory where datanode store data.
4. Then check which Java processes are running with `jps`. You should see : Jps, DataNode, NameNode, SecondaryNameNode, NodeManager, ResourceManager. You can also check the other processes (i.e., including non-java) with `ps -ef` but there shouldn't be many more.

### 1.2 Managing files with HDFS

1. Move the text file `words.txt` from your computer to the container. `docker cp words.txt hadoop-lab:/`.
2. Create a (distributed) directory in hdfs `bin/hdfs dfs -mkdir /rep`[1].
3. Copy the file from the local file system to hdfs : `bin/hdfs dfs -put /words.txt /rep/words.txt`.
   You can then check with `bin/hdfs dfs -ls /rep` and `bin/hdfs dfs -tail /rep/words.txt` (file is a bit large for `cat`)
   Check also in the local filesystem the content of the directory where the datanode stores its data.

   > You should mostly observe an arborescence built around a path of the form /opt/hadoop/dfs/data/current/BP-.../current/finalized/subdir0/subdir0/...
   > With namenode federation several namenodes may share the datanode, hence each namenode gets its own namespace. A block pool is a set of blocks that belongs to a same namespace (those blocks are distributed across Datanodes which thus may contain blocks from multiple block pools - potentially all block pools in the cluster). Each block pool is identified uniquely by some unique ID BP-<namespaceID>-<namenodeIP>-<creationTime> (though an UUID would do as well since this is not parsed by HDFS: the mapping from a block pool to its namespace being stored in the VERSION file, and the mapping from namespaces to namenodes in the hdfs-site.xml on the DataNode, but here we have a single namespace so the hdfs-site.xml is simpler.)
   > You probably can see also a "rbw" directory but this one is for *replica being written*, hence empty, and likewise a "tmp" one.

[1] Here and below, you may alternatively use `bin/hadoop fs` instead of `bin/hdfs dfs` but the latter is more efficient since these are instructions dealing with hdfs and not an arbitrary file system.

## 2 Hadoop Streaming: running a MapReduce job defined with executables

https://hadoop.apache.org/docs/current/hadoop-streaming/HadoopStreaming.html

Our purpose in this section is to execute a mapreduce job defined with arbitrary executables. First with shell scripts, then with two python scripts; `mapper.py` and `reducer.py`.

4. Let's first run the Hadoop Streaming utility with a Java class: `org.apache.hadoop.mapred.lib.IdentityMapper` and a bash scripts: `/usr/bin/wc`.

```
bin/hadoop jar share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar \
  -input /rep \
  -output outputdir0 \
  -inputformat org.apache.hadoop.mapred.KeyValueTextInputFormat \
  -mapper org.apache.hadoop.mapred.lib.IdentityMapper \
  -reducer /usr/bin/wc
```

> The output directory will be created by the Job. It should not pre-exist.

If this fails by getting stuck with a message like "Running job: job_xxxx" and nothing happens for more than 1 minute, I just don't know how to fix that (it's a rather old image, people nowadays would rather use the bde2020 image, but it requires docker compose). In that case just skip this one question but you can still answer the following questions as you can simulate mapreduce and its shuffle step using a unix pipe, as we discuss next.

**Question 1:** What is the result of this job ?

5. > It's a good idea to check locally your pipeline before running it on map reduce, with the instruction below. A priori, if you restricted yourself to the instructions above, you probably cannot run the pipeline, neither on the namenode (no Python) nor on nodemanager (no files), but you can easily run the pipeline on your own computer outside of docker, or use any a container with both python and the scripts (adapting one of the above or creating another one).
> `echo 'we know what we are, but not what we may be' \`
> `| ./mapper.py | sort -t 1 | ./reducer.py`
> Of course, use `cat` instead of `echo` when the input is a file.
> When testing with a unix pipe, it is better to use the reduced file miniword, which only contains words starting with an "a", that can be obtained using `sed -Ei.old '/.* [b-zB-Z]/d' miniwords.txt`

   If your job got stuck as described in the previous question, just run this job locally with a unix pipe (on the reduced dataset), as discussed above. Otherwise, put the Python scripts `mapper.py` and `reducer.py` on the namenode (in our case, that just means on the container). Make sure they are executable (otherwise : `chmod a+x mapper.py`...)

   Then execute a MapReduce job that uses these mapper and reducer, adapting the instructions below if needed :

```
bin/hadoop jar share/hadoop/tools/lib/hadoop-streaming-2.7.0.jar
  -input /rep\
  -output outputdir1 \
  -inputformat org.apache.hadoop.mapred.KeyValueTextInputFormat \
  -mapper /mapper.py \
  -reducer /reducer.py
```

## 3 Using a python library such as mrjob

You will probably find it more convenient to run a mapreduce job using the mrjob library. Create an input directory in hdfs, put the file lai-eliduc.txt in it. Then Understand and run the words_cout_with_mrjob.py example using `python3 /opt/hadoop/words_countwith_mrjob.py -r hadoop hdfs:///input/input.txt -o hdfs:///output` and check the result.

## 4 Write your own MapReduce job

The file `words.txt` provided with these instructions contains words from multiple langages[2]. Each line is a pair: `language word` as illustrated below. Neither word nor language contain any space:

[2]This file was obtained by joining files from https://github.com/lorenbrichter/Words/tree/master/Words

```
deutsch Aachen
français abaque
english aardvark
...
```

**Question 2:** Write a Job that processes the file `words.txt` to compute the words that appear at least once in both french and english (each word should of course appear once only in the result). This means you are computing the intersection of the two languages. Your program should write the result as a list of pairs (w,0) where w is a word that appears in both french and english.

- Your Job should be efficient even with much larger files (be reasonably clever about your map and reduce. Don't just purposedly send the whole data to one single reducer).
- Indication: you will have to deal with one non-ascii caracter. This is not an issue with Python3. But if you were using python2, the header of your python file should probably be :

  ```
  #!/usr/bin/env python
  # -*- coding: utf-8 -*-
  ```

- Ideally you should not assume that each word appears only once in each language. Your program should able to accept a word x that occurs twice with "français" and once with "english", for instance.